

FinBot: Multilingual Financial Assistant

Tuong Vy Vu

Technical Report

FinBot: A Smart Multilingual Financial Assistant

FinBot is a multilingual financial assistant that collects a user’s financial profile through a structured dialogue and generates a tailored recommendation for Planning, Investment, or Trading. The system is built around a locally deployed LLM, a deterministic dialogue engine, and a browser-based frontend, with the design deliberately optimized for reliability, privacy, and practical local deployment rather than only raw model size.

This report presents the complete system design, implementation decisions, evaluation results, and justification for the final architecture. The report is also written to directly address the design criteria: model comparison and deployment, dialogue automation, frontend design, recommendation quality, experimentation with techniques, and higher-mark advancements such as multilingual support, teacher-student distillation, and prompt-injection defense.

1. Project Goals

The core goal of the project was to build a smart personal financial assistant that can interact with users, collect relevant financial information, and generate customized financial strategies using a local LLM. This directly matches the requirement to create an LLM-powered assistant for personalized finance-related recommendations rather than a generic chatbot.

A second goal was to make the system robust enough for real use under local hardware constraints. That requirement shaped almost every design choice in the project: the selected model had to be small enough to run on consumer devices, the dialogue system had to minimize unnecessary LLM calls, and the output had to be validated so that the frontend could safely render it without breaking.

A third goal was to go beyond a baseline implementation and address the “higher design” criteria in the system. FinBot therefore includes four advanced contributions: multilingual support across English, Vietnamese, and Chinese; a deterministic one-LLM-plus-FSM architecture; teacher-student sequence-level knowledge distillation with measurable gains; and multilingual prompt-injection hardening integrated into the dialogue pipeline. These features were not added as extras after the system was finished; they were built into the architecture from the beginning because they improve both usability and technical quality.

2. Model Selection and Comparison

Model selection was treated as an evidence-based engineering decision rather than a preference-based choice. Three small open-weight instruction-tuned models were shortlisted: Qwen2.5-0.5B-Instruct, Qwen2.5-1.5B-Instruct, and gemma-2-2b-it. These three models were appropriate candidates because they are open-weight, instruction-tuned, practical for local deployment, and sufficiently different in size to reveal the trade-off between efficiency and generation quality.

The evaluation protocol was intentionally simple and reproducible. Each model was tested on the same four multilingual finance prompts under deterministic decoding, using the same system prompt and the same scoring rubric. The rubric weighted five dimensions: instruction following, financial quality, safety, language fidelity, and structured output. This matters because choosing a model for this system is not only about whether it can answer a finance question, but whether it can answer in the right language, in the expected format, and under strict safety constraints.

Model	Avg Weighted	Instruction	Finance	Safety	Language	Structured
Qwen2.5-1.5B-Instruct	4.47	5.00	4.00	3.75	5.00	4.50
Qwen2.5-0.5B-Instruct	4.40	5.00	4.00	3.75	5.00	4.00
gemma-2-2b-it	4.38	5.00	4.75	4.00	3.50	4.50

Qwen2.5-1.5B-Instruct was selected as the final backbone because it achieved the highest weighted score, preserved multilingual fidelity, and remained small enough to fit the local deployment budget. The most important practical advantage over Gemma was language reliability: Gemma achieved only 3.5 on language fidelity because it sometimes collapsed into English when prompted in Vietnamese or Chinese, whereas Qwen2.5-1.5B maintained a perfect 5.0 language score. For this project, that difference was more important than Gemma’s slightly stronger finance score, because multilingual consistency is central to the advancing system criteria.

The 1.5B size was also a more balanced engineering choice than the 0.5B alternative. Although the 0.5B Qwen model came close in weighted score, the 1.5B version offered stronger structured output quality and more headroom for LoRA adaptation while still remaining realistic for Apple Silicon or consumer CUDA hardware. In other words, the selected model was not simply the best on average; it was the best match for the project’s combined constraints of multilingual accuracy, structure control, and local inference.

3. Overall System Architecture

FinBot uses a deliberately hybrid architecture: one local LLM for final reasoning, and deterministic software components for everything that does not require generative intelligence. The stack consists of a FastAPI backend, a finite-state dialogue engine, a browser frontend built with vanilla JavaScript and Bootstrap 5, and a Hugging Face-served Qwen2.5-1.5B model with an optional LoRA adapter.

The most important architectural decision was to avoid a multi-LLM agent design. Instead of having one model plan the dialogue and another model produce the answer, the system uses a deterministic finite-state machine for information collection and reserves the LLM call for the final recommendation only. This choice was made for two reasons. First, a finite-state machine is more reliable than a second LLM for slot filling because it does not hallucinate fields, drift off task, or consume unnecessary tokens. Second, a second model would roughly double memory cost and noticeably increase latency, which would be hard to justify in a system that is meant to run on local consumer hardware.

This architecture can be summarized as “cheap deterministic logic first, one expensive smart call at the end.” That principle is especially effective when the model budget is only 1.5B parameters. The system gathers structured inputs using rules and multilingual parsers, validates what is collected, and only then asks the LLM to perform the part of the task that actually benefits from generative reasoning: transforming a profile into a tailored financial recommendation.

The architecture also improves maintainability. Each responsibility is clearly separated:

`state_machine.py` governs interaction flow, `task_policy.py` defines which fields are required for each task, `prompt_builder.py` constructs the final instruction package, `safety.py` handles sanitization and redaction, `recommender.py` validates and repairs model outputs, and `llm_adapter.py` manages loading and inference.

This separation makes the system easier to debug, evaluate, and explain in a report, which is better for system design, implementation, and core code discussion.

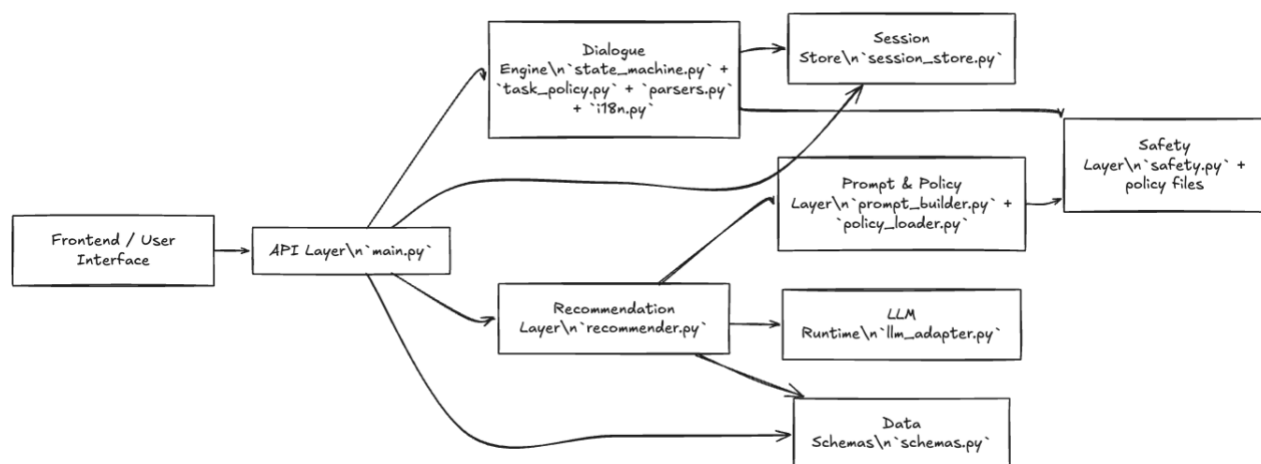


Fig.1 Overall Architecture

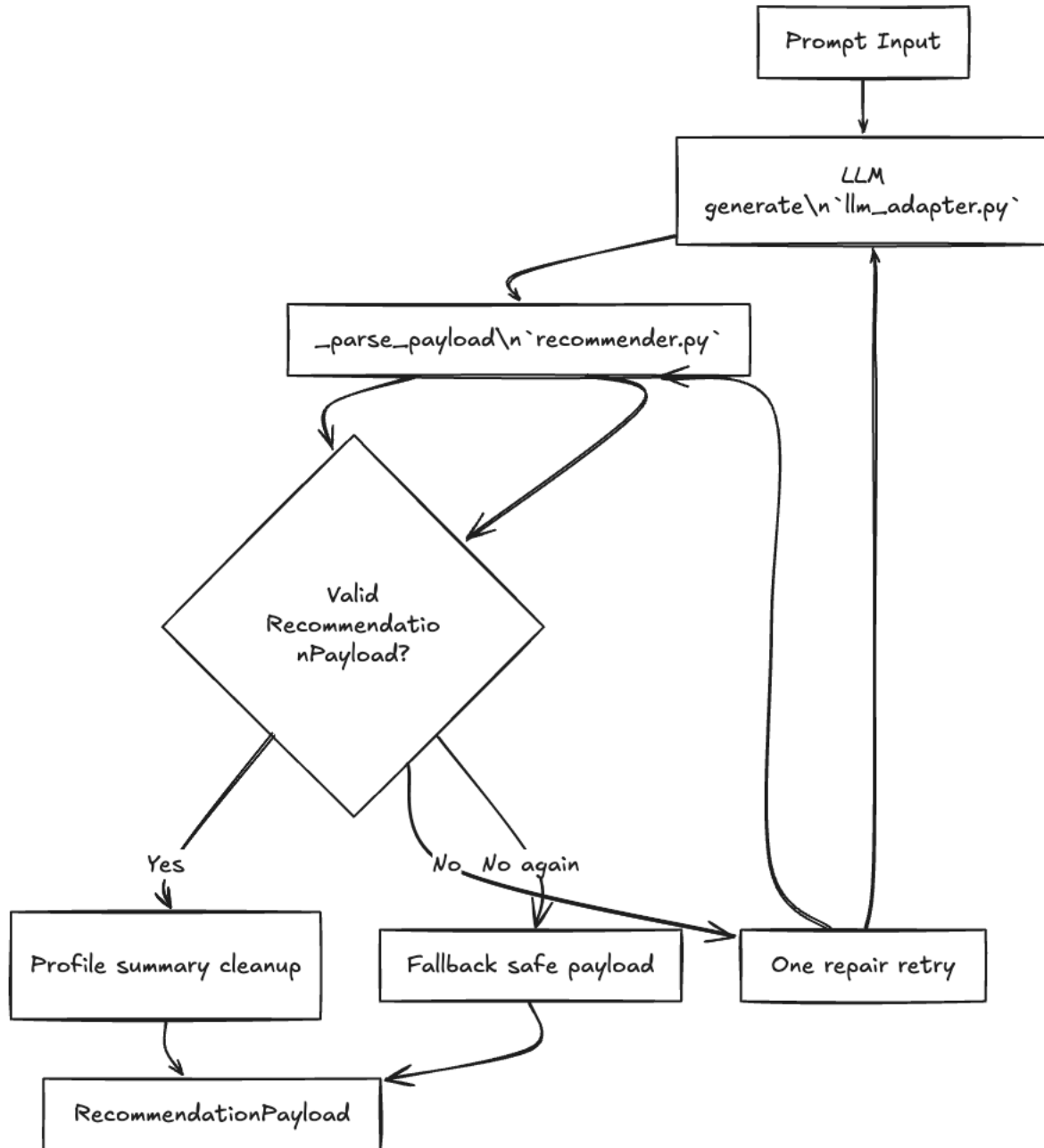


Fig. 2 Recommendation Pipeline

4. Dialogue Design and Information Collection

The dialogue system is implemented as a deterministic finite-state machine that moves the user from an ASKING state to a RECOMMENDING state. At the beginning of a session, the system stores a new UUID-linked session record with an empty profile and `next_item = "TASK_MODE"`. From that point onward, each user turn is interpreted according to the current state and the next missing field.

This design ensures that the system collects information in a controlled and explainable way. Rather than asking broad or repetitive questions, the system gathers one field at a time, validates the answer, and decides what to ask next. That makes the interaction easier for users and also easier to evaluate, because the system's behavior is reproducible. This is an important strength relative to fully free-form conversational agents, where the collection of required financial information can become inconsistent from one user to the next.

The task mode determines which information must be collected:

Task Mode	Required Fields
Planning	GOAL, INCOME_BAND, CAPITAL_RANGE, TIME_HORIZON, RISK_TOLERANCE
Investment	GOAL, CAPITAL_RANGE, TIME_HORIZON, RISK_TOLERANCE
Trading	GOAL, CAPITAL_RANGE, RISK_TOLERANCE, TIME_HORIZON

This per-task policy is important because it shows that the assistant adapts to the user's request rather than forcing the same template onto every interaction. For example, `INCOME_BAND` is required only for Planning, where savings capacity matters, but is not requested for Investment or Trading. This is both a privacy improvement and a task-design improvement, because the system avoids collecting irrelevant personal data.

The system also handles imperfect interaction gracefully. Each field allows one clarification retry if the answer cannot be mapped. If the second attempt still fails, the system records the field as `UNKNOWN` instead of blocking the dialogue entirely. The recommendation gate permits generation when no more than half of the required fields are unknown, which prevents the user from getting stuck in a dead-end conversation while still preserving a minimum amount of usable context.

5. Multilingual and Privacy-Aware Parsing

A major contribution of FinBot is that multilingual support is not limited to final output generation. The system supports multilingual interaction throughout the whole pipeline: task selection, field

input, clarification prompts, side-panel UI labels, and final recommendation generation. English, Vietnamese, and Chinese are handled consistently in both the frontend and backend.

This is achieved through several coordinated components. `i18n.py` stores assistant-facing strings in all three languages, while `parsers.py` normalizes multiple answer formats into internal enums. A user can answer with a number, an English term, a Vietnamese phrase, or a Chinese phrase, and the system maps all of them into the same canonical representation. For example, task choices and risk labels are normalized before they are stored, which makes later prompting and training much more stable.

This normalization strategy also strengthens privacy. Sensitive information is not passed to the model as highly specific raw values where possible. Instead, financial attributes such as income and capital are bucketed into coarse bands like "60-120K" or "MEDIUM". This protects users by reducing exposure of exact financial details and also helps model generalization, because the fine-tuning data becomes more regular and less sparse.

The language experience is further improved by automatic detection. `state_machine.detect_language` uses Vietnamese diacritics and CJK Unicode ranges to infer whether the user is writing in Vietnamese or Chinese; otherwise, the system defaults appropriately. This is a practical feature because it allows the assistant to adapt to the user's language behavior directly, rather than requiring the user to manually configure everything before interaction begins.

6. Prompt Design

The production prompt is one of the most important technical elements of the system. It is not a simple generic instruction asking the model to "be a financial assistant." Instead, it is a highly engineered structured prompt that combines role conditioning, a constrained reasoning protocol, direct schema injection, strict negative rules, and task-specific guidance.

At the center of the prompt is the "Quantitative Execution Protocol." This protocol forces the model to follow a shape-constrained reasoning process: extract and normalize numbers, compute the financial delta, apply task-specific logic, stress-test at negative 20 percent, and then produce a final output with formula, substitution, and result. This design was chosen because sub-2B models often fail when given vague free-form reasoning prompts. A smaller model benefits much more when the structure of reasoning is explicitly dictated.

The prompt also injects the exact JSON schema from `RecommendationPayload.model_json_schema()`. This is a major design choice because the model's output is not consumed as plain text; it must be parsed

and rendered by the frontend. Embedding the schema directly in the prompt helps the model internalize the exact target structure, which later becomes even more effective after fine-tuning on demonstrations that follow that same structure.

Negative constraints are equally important. The prompt explicitly forbids vague language, markdown wrappers, and motivational filler, and requires concrete execution verbs such as “Allocate,” “Execute,” and “Set.” That stylistic control is not merely cosmetic. In a finance assistant, vague wording reduces usefulness, while excessive prose increases output length and often harms JSON validity. The prompt therefore acts as both a reasoning scaffold and a compression mechanism.

7. Safety and Prompt-Injection Defense

Because FinBot is a financial assistant, safety is not optional. The system implements defense-in-depth safety rather than relying on a single refusal instruction inside the prompt. This includes privacy rules, refusal topics, output rules, input sanitization, and output-level leakage detection.

At the input level, `safety.py` redacts personally identifiable information such as emails and phone numbers from free-text fields. This is especially important for the GOAL field, because users may casually include details that are not necessary for recommendation generation. The system therefore reduces risk before the LLM ever sees the text.

The project also implements multilingual prompt-injection sanitization. Instruction-override patterns are detected and sanitized in English, Vietnamese, and Chinese. This defense is applied in at least two places before generation: during GOAL capture in `state_machine.py`, and again in `main.py` across all collected fields before prompt assembly. This layered approach matters because malicious or off-task instructions can appear in different parts of the dialogue, not only in the first user message.

At the output level, `recommender.py` checks for multilingual meta-instruction leakage, such as references to the system prompt itself. If such content appears, the output is rejected and the system can retry. This closes an important loop: the system not only sanitizes inputs, but also prevents internal instructions from leaking back to the user. This is a strong example of an “advanced LLM technique” that directly supports the requirement for more robust financial-assistance systems.

8. Recommendation Generation and Validation

The final recommendation must be both useful and machine-readable. To achieve this, FinBot requires every recommendation to conform to a strict `RecommendationPayload` schema with fields for

`profile_summary`, `recommendation`, `reasoning`, `risks_caveats`, `sources`, and `disclaimer`. This schema is central to both the frontend rendering logic and the fine-tuning target format.

However, small local models are prone to formatting errors such as markdown wrappers, malformed braces, or partial JSON. For that reason, FinBot does not trust the first model output blindly. Instead, `recommender.py` runs a cascading validation pipeline: direct Pydantic parsing, raw decode scanning, and brace-slicing extraction. If all three fail, the system performs a single repair retry by feeding the bad output back into the model and explicitly asking for corrected JSON.

This validation-and-repair pipeline is one of the key engineering decisions that made the system practical. Without it, frontend rendering would be fragile, and model errors would become user-facing failures. With it, many common small-model mistakes can be recovered automatically. The design also remains honest about its current limits: semantic checks, such as verifying whether the model actually used a formula or performed sufficient numeric reasoning, are still future work. At present, the pipeline guarantees syntactic validity more strongly than semantic correctness.

Even so, after fine-tuning, the importance of the repair loop decreases because the model itself becomes much better at emitting the correct schema. That makes the validation layer a safety net rather than the main mechanism enforcing structure, which is the preferable system design.

9. Local LLM Configuration and Deployment

The model serving layer is implemented in `llm_adapter.py`, which loads the tokenizer and model, optionally attaches a LoRA adapter, and exposes a Hugging Face text-generation pipeline. Serving is deterministic, with `do_sample=False`, a repetition penalty, and a controlled token budget. This deterministic configuration was chosen to make evaluation reproducible and to reduce unpredictable variance in outputs.

A technically important detail is the use of `tokenizer.apply_chat_template(..., add_generation_prompt=True)`.

Qwen2.5-Instruct was trained with ChatML-style boundaries, so using the native template aligns the inference format with the model’s training format. The report context explicitly notes that hand-written “System/User” string formats produce worse instruction following and occasional JSON failures, especially at this model scale.

Deployment is environment-driven. The system can switch models and adapters using environment variables such as `FINBOT_BASE_MODEL`, `FINBOT_ADAPTER_PATH`, and `FINBOT_ADAPTER_MODEL_ID`. This

makes experimentation and A/B comparison easy without code changes. It also improves portability between development and demonstration environments, which is valuable for a course project that must be packaged and presented clearly.

The project deliberately ships only the LoRA adapter instead of a merged checkpoint. That makes CUDA-to-MPS migration much simpler: the adapter is lightweight, device-agnostic, and can be attached to the base Qwen model at runtime. On Apple Silicon, the system loads the base in fp16 using MPS and attaches the adapter on top, which keeps memory requirements manageable at roughly 3 GB plus a small LoRA overlay.

10. Data Collection and Dataset Design

To support supervised fine-tuning, the project constructed a multilingual dataset of financial recommendation examples stored as JSONL files. The raw corpus covers three tasks—planning, investment, and trading—across three languages—English, Vietnamese, and Chinese—for a total of nine files and approximately 621 raw records.

Each record includes a bucketed profile, a set of unknown fields, and a structured response target containing summary, recommendation, reasoning, risks, sources, and disclaimer. This format is important because it mirrors the live system’s final output contract. Rather than training the model on general finance text, the project trained it on exactly the type of structured financial assistance it was expected to produce during inference.

The data-cleaning pipeline was deterministic and explicit. According to the report context, 618 raw examples were processed, with 615 kept after filtering; zero failed schema validation, two were removed for unsafe content, one duplicate was removed, and none were dropped for missing task or language once those could be inferred from filenames. This is a strong data story for the system because it shows that the training corpus was curated rather than assembled carelessly.

The cleaned dataset was also reasonably balanced. The final counts were 210 planning, 196 investment, and 209 trading examples, with language counts of 208 English, 208 Vietnamese, and 199 Chinese. The train/eval split was stratified on task mode and language, resulting in 553 training examples and 62 held-out evaluation examples. This balanced design helps justify the later claim that the fine-tuned model performs consistently across tasks and languages rather than overfitting to one dominant subgroup.

11. Fine-Tuning Through Teacher-Student Distillation

The project’s most important advanced technique is sequence-level teacher-student knowledge distillation. Two frontier teacher models, GPT-5.3 and Gemini-3-Flash, were used to generate high-quality structured responses. These teacher outputs became the supervision targets for Qwen2.5-1.5B, which served as the student model.

This was a particularly suitable technique for the project because the main weakness of the base local model was not broad language understanding, but reliable structured output and explicit quantitative reasoning. The teachers already produced valid RecommendationPayload JSON and populated reasoning traces. By training on those demonstrations, the student could internalize the exact response pattern needed by the application.

The fine-tuning run used LoRA with `r = 16`, `lora_alpha = 32`, and dropout 0.05, targeting attention and MLP projection layers. Only 18,464,768 parameters were trainable, which is about 1.182 percent of the 1.56B backbone. Training took place on an NVIDIA A100-SXM4-80GB GPU and completed in 156 steps, approximately 3.0 minutes, using fp16 LoRA and packed 2048-token sequences.

The loss curve reported in the context is highly consistent with successful structure learning. Training loss dropped from 1.853 at step 10 to roughly the 0.22–0.26 range by the end of training, with the sharpest improvement in the first 30 steps. This pattern strongly suggests that the model quickly learned the JSON scaffold and then spent the remainder of training refining the structure and style of the response.

12. Fine-Tuning Results

The effect of fine-tuning was large and directly measurable on the held-out set of 62 examples. This is the strongest empirical evidence in the entire report because it shows that the adaptation was not cosmetic; it solved the most important practical deployment problem.

Metric	Base Qwen2.5-1.5B	+ FinBot LoRA
schema_valid_rate	0.000	1.000
safety_pass_rate	1.000	1.000
internal_analysis_present_rate	0.000	1.000
mean_output_tokens	638.13	168.19
mean_latency_ms	5171.8	3347.2

The most important result is the jump in schema-valid rate from 0 percent to 100 percent. Before fine-tuning, the base model often produced free-form or malformed JSON that failed validation. After fine-tuning, every held-out output validated successfully. For a system whose frontend depends on strict structured output, this is arguably the single most important metric in the project.

The second major result is that the reasoning field, which acts as the internal analysis trace, also improved from 0 percent populated to 100 percent populated. This means the model did not merely learn to emit the correct outer shape; it learned to fill the reasoning component in the teacher-demonstrated “because data → formula → delta → action” style.

The gains were not achieved by producing longer answers. In fact, the opposite happened: mean output length dropped by about 74 percent, from 638.13 tokens to 168.19 tokens, and latency fell by about 35 percent, from 5171.8 ms to 3347.2 ms. This shows that the fine-tuned model became not only more accurate in structure, but also more concise and efficient. Safety remained at 100 percent, so the performance improvement did not come at the cost of harmful recommendation behavior.

13. Cross-Lingual Performance

A strong multilingual claim requires more than a multilingual training set; it also requires evidence that the model behaves consistently across languages. The held-out evaluation in the report context includes per-language breakdowns for English, Vietnamese, and Chinese.

For all three languages, the base model achieved 0.000 schema-valid rate and 0.000 internal-analysis-present rate, while the fine-tuned model achieved 1.000 on both metrics with safety preserved at 1.000. Specifically, the held-out set contained 21 Vietnamese, 20 Chinese, and 21 English examples, and the fine-tuned model succeeded on every subgroup.

This result is significant because multilingual systems often degrade unevenly across languages even when the aggregate metric looks strong. In FinBot, the same before/after pattern appears in each language, which suggests that the distillation approach transferred the structured response behavior across the multilingual space rather than only memorizing English formatting patterns. This is one of the strongest arguments that the system satisfies the request to support multiple languages in a meaningful way.

14. Prompting Ablation and Technique Comparison

In addition to comparing models and comparing weights before and after fine-tuning, the project also investigated different prompting settings. A four-arm ablation study evaluated `baseline_structured`, `few_shot_structured`, `brief_reasoning`, and `self_check_instruction` under deterministic decoding.

Technique	Schema Pass	Safety Pass	Relevance	Mean Latency (ms)
<code>baseline_structured</code>	1.000	1.000	0.000	3361.0
<code>few_shot_structured</code>	1.000	1.000	0.000	4775.5
<code>brief_reasoning</code>	1.000	1.000	0.000	7115.5
<code>self_check_instruction</code>	1.000	1.000	0.000	9425.5

The primary conclusion from this ablation is that all four prompt variants were sufficient to preserve schema and safety under this particular run, but they had very different latency costs. `baseline_structured` was clearly the fastest, while self-checking nearly tripled response time. This matters for deployment because prompt engineering is not free; each extra layer of reasoning or demonstration adds latency that may not be justified if the LoRA adapter already guarantees strong structure.

The `relevance_score` remained zero for all techniques, but the report context explains that this is due to a strict evaluation heuristic rather than actual semantic failure. That limitation should be reported honestly. The key value of the ablation is therefore not the relevance metric, but the insight that once the model has been properly fine-tuned, a strong structured baseline prompt can achieve the needed behavior much more efficiently than heavier prompt variants.

15. Frontend Design and User Experience

The frontend was built as a real website rather than a notebook UI. It uses `frontend/index.html` and `frontend/app.js`, styled with Bootstrap 5 and custom CSS, and presents the interaction in a clean chat-based layout.

Several interface elements were designed specifically to make the system easier to understand and demonstrate. The chat area shows the conversation and the final recommendation card. A side panel shows live system state, including the current task mode, the next required field, the collected profile, unknown fields, and readiness for recommendation. This makes the dialogue system visible rather than hidden, which is especially helpful for explaining the system in the report and the video demo.

The frontend also supports practical interaction features expected from a polished assistant: language selection, model selection, session persistence via local storage, reset controls, and typing-style feedback. These features do not directly improve LLM reasoning, but they significantly improve the usability and professionalism of the system. This layer contributes meaningfully review, and also rewards a friendly frontend and justified presentation.

The recommendation card itself is structured to match the backend schema. It separates profile summary, recommendation, reasoning, risks and caveats, sources, and disclaimer into distinct visual sections. This is an important UI choice because financial guidance needs to feel organized and auditable rather than like a wall of unstructured text.

16. Demonstrating a Working Dialogue System

The dialogue API is exposed through `POST /chat/turn`, which accepts a session ID, user message, model identifier, and optional language hint, and returns the system state, task mode, next item, collected fields, unknown fields, readiness flag, recommendation, and meta information. This contract demonstrates that the dialogue system is not ad hoc; it is a formal interface between frontend and backend.

A working interaction therefore proceeds in a predictable sequence. The system first identifies or confirms the task mode, then collects the required financial attributes for that task, then passes the final structured profile to the recommendation generator. This staged interaction build an LLM-driven dialogue that collects personal and financial information before producing a tailored recommendation.

The system is also easier to demonstrate than a free-form chatbot because the dialogue behavior is inspectable. During a live demo, the marker can see the state machine progressing, the profile fields being stored, and the recommendation only appearing when the gating conditions are met. That transparency helps show that the system is genuinely designed rather than just prompt-wrapped.

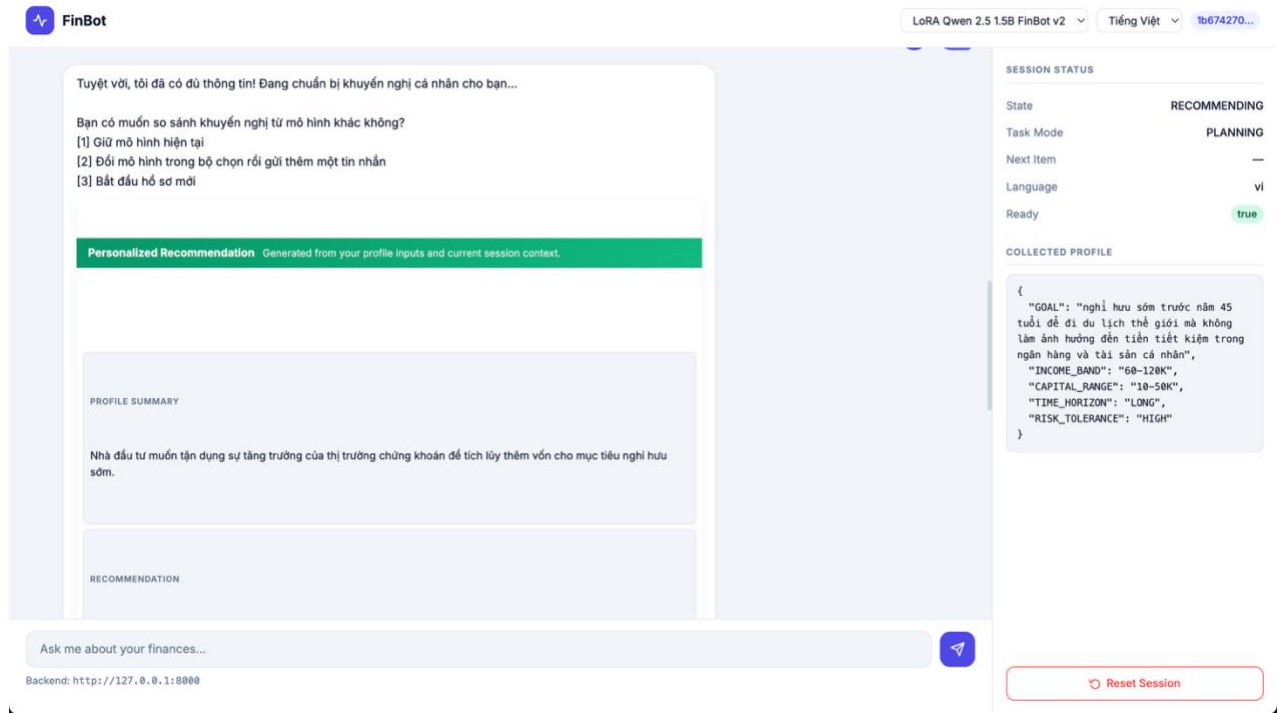


Fig 3. Planning Vietnamese 1

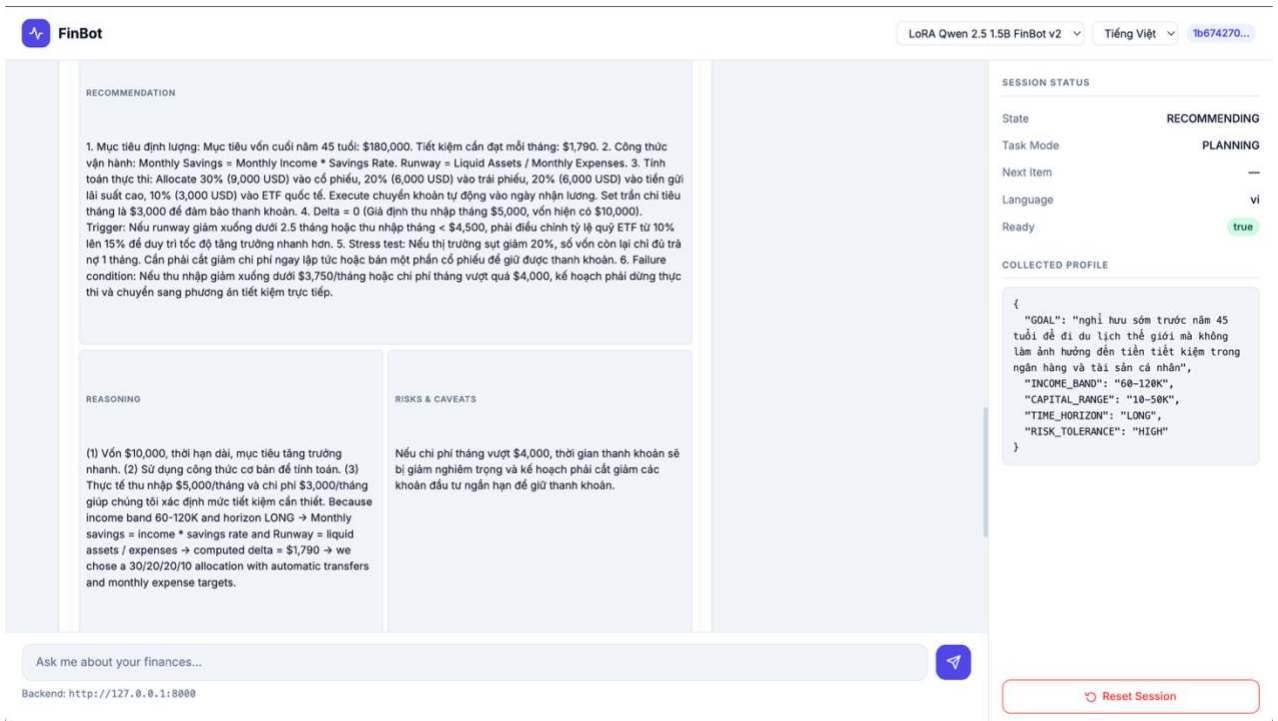


Fig 4. Planning Vietnamese 2

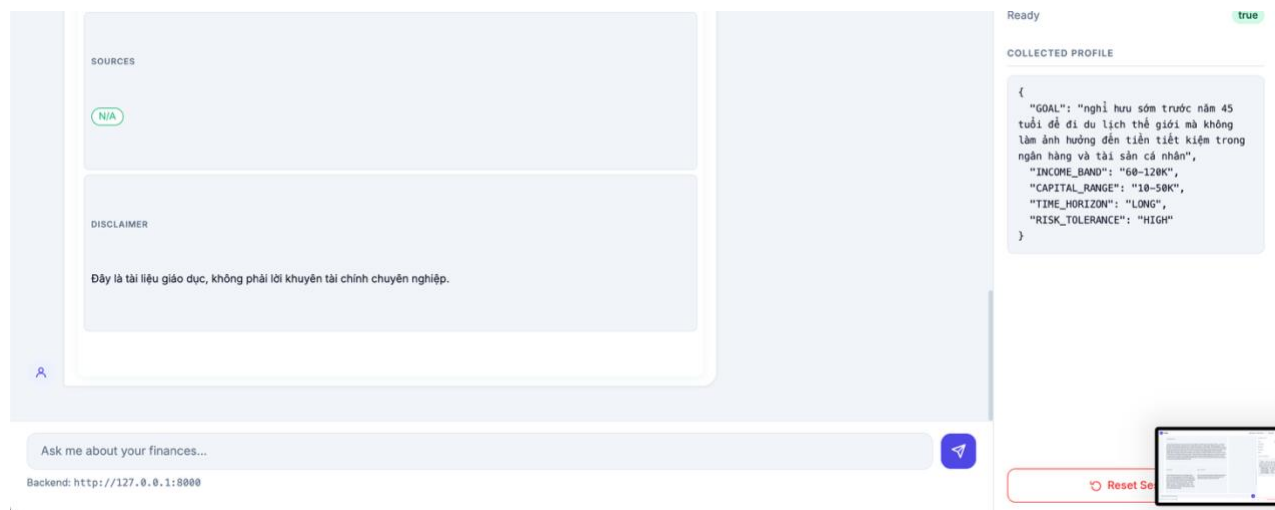


Fig 5. Planning Vietnamese 3

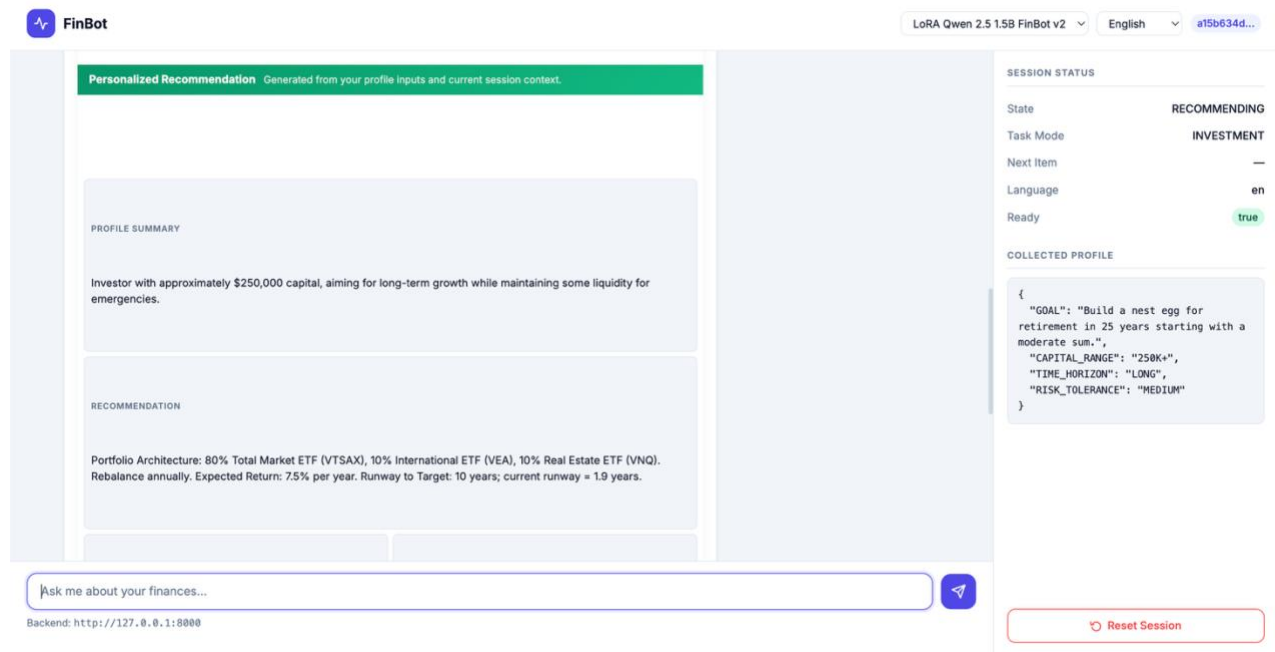


Fig 6. Investment English 1

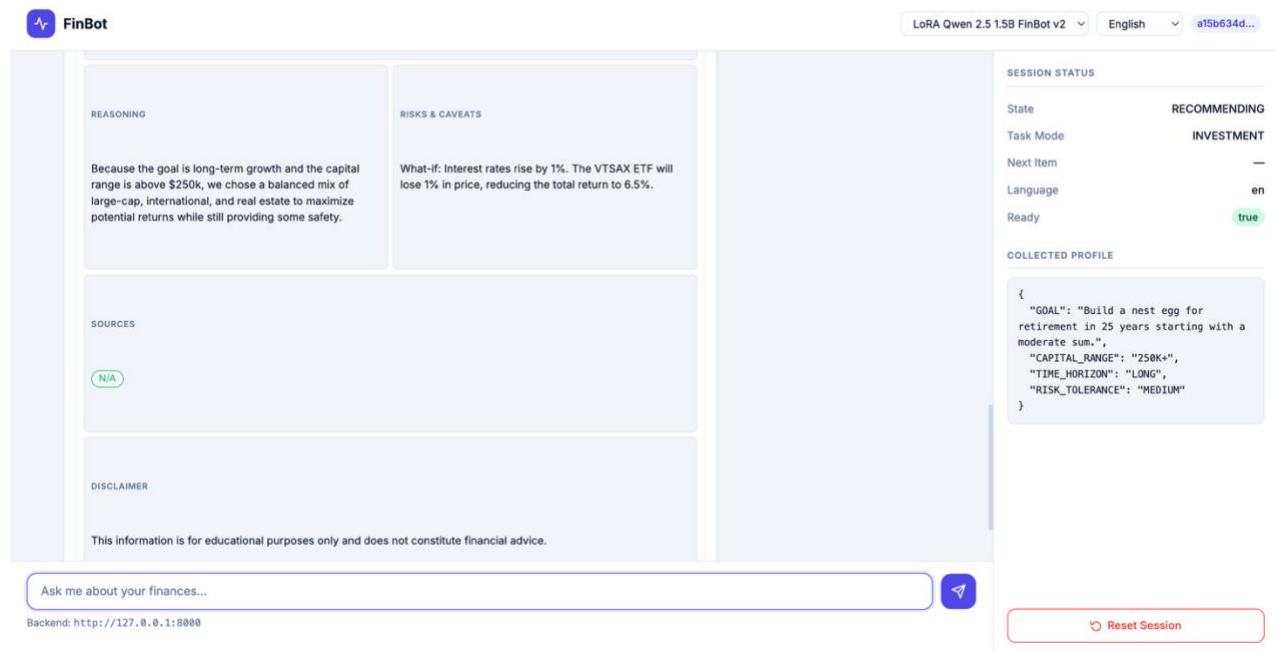


Fig 7. Investment English 2

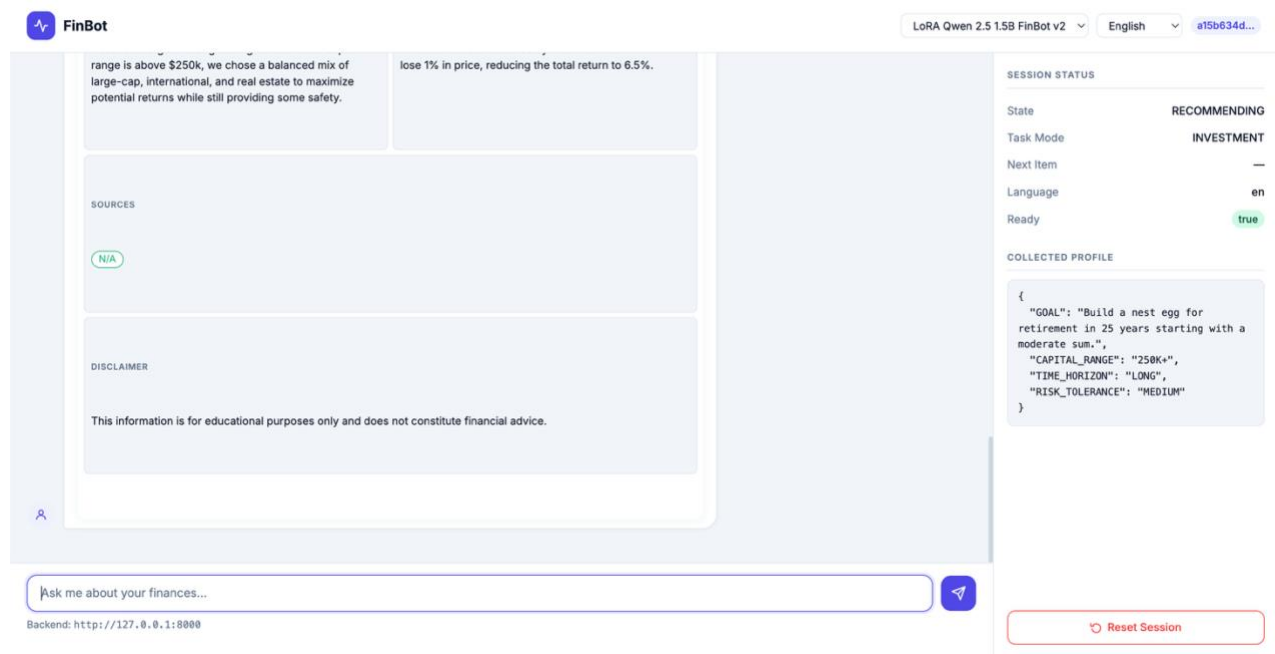


Fig 8. Investment English 3

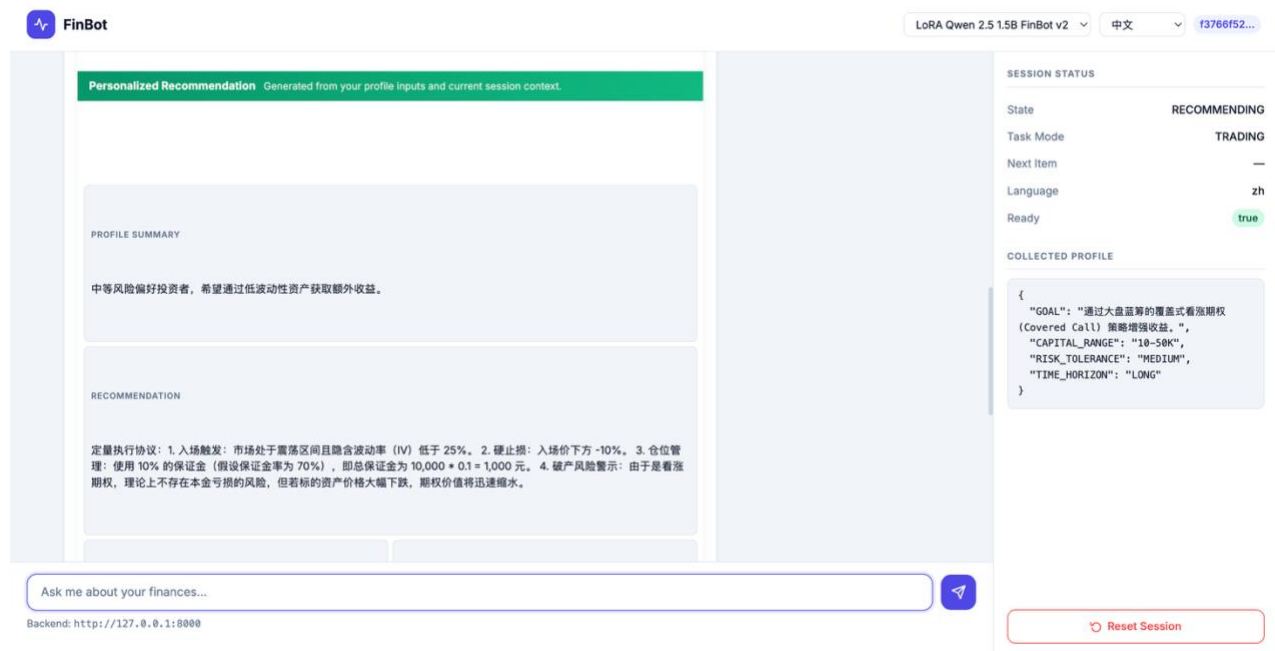


Fig 9. Trading Simplified Chinese 1

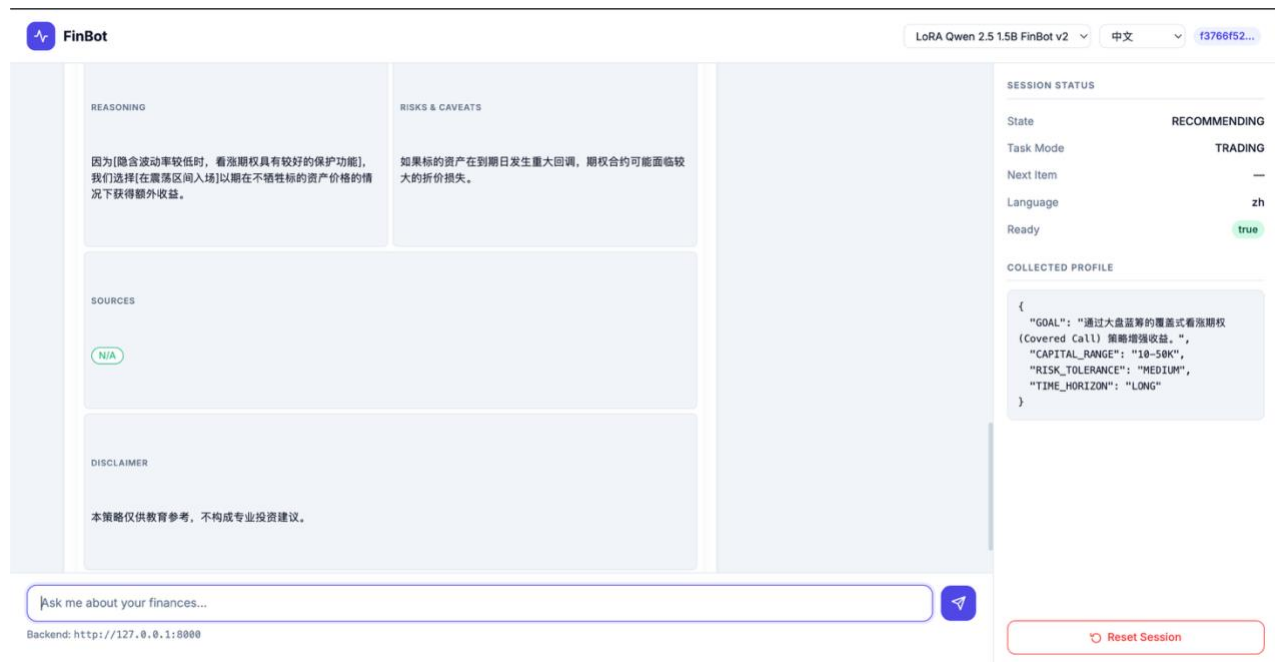


Fig 10. Trading Simplified Chinese 2

17. Limitations and Future Work

Despite the strong results, the current system still has clear limitations. The most important one is that recommendation validation is primarily syntactic rather than semantic. The system can confirm that the output is valid JSON and contains the required fields, but it does not yet robustly verify that the numeric reasoning is truly correct, that a formula was applied properly, or that enough quantitative evidence was used.

A second limitation is that the current retrieval capability is effectively stubbed. The report context notes that RAG hooks exist but `meta.used_rag = False`, which means recommendations are generated from the user profile and prompt design rather than live market data, announcements, or external financial sources. Adding lightweight retrieval would improve factual grounding for time-sensitive financial contexts.

A third limitation is scalability. Session state is currently stored in-memory, which is fine for local demonstration and single-process deployment but not sufficient for production-grade concurrency. Moving the session layer to Redis would be the natural next step if the system were expanded beyond the proof of concept.

Finally, Apple Silicon deployment currently relies on fp16 rather than 4-bit quantization because `bitsandbytes` QLoRA is CUDA-only. The current deployment is still practical, but future optimization could further reduce memory and latency if broader cross-platform quantization tooling becomes available.

18. Conclusion

FinBot demonstrates that a small local LLM can be turned into a reliable multilingual financial assistant when it is embedded inside a carefully engineered system. The project's main insight is that model quality alone is not enough; performance emerges from the interaction between model selection, deterministic dialogue design, structured prompting, validation, and targeted fine-tuning.

Among all contributions, the most important is the teacher-student LoRA adaptation of Qwen2.5-1.5B. It transformed the model from a structurally unreliable generator with 0 percent schema-valid outputs into a fully compliant assistant with 100 percent schema validity, 100 percent populated reasoning traces, shorter outputs, and lower latency on the held-out evaluation set. That result, combined with multilingual support and a polished frontend, gives the project a great claim to the top PoC system.